LabSync: Live System Security Verification Guide

Group Demonstration & Examination Testing Script for Live Deployed Application

Live Railway Deployment: `https://labsync-bulsu-sarmiento-production.up.railway.app`

CATEGORY A: LOGIN AND ACCOUNT SECURITY

#1 Account Provisioning, Session Checks & Public Kiosk Access — RETAIN

Target URL: `/api/schedules/save` | `/public/report.html`

How to Test:
1. Open an Incognito/Private window.
2. Paste `https://labsync-bulsu-sarmiento-production.up.railway.app/api/schedules/save` directly into the address bar.
3. Next, open `/public/report.html` in the same incognito window.

Expected Result: Protected API returns `HTTP 401 Unauthorized` (access blocked). Meanwhile, the student fault-report page opens cleanly without requiring any login.

What to say: *"Our server requires sessions on staff routes, but public student fault reporting is intentionally accessible so students at kiosks aren't blocked by login."*

#2 Credential Recovery: 32-Byte Cryptographic Tokens & 1-Hour Expiry — RETAIN

Target URL: `/recover-password.html`

How to Test:
1. Open `/recover-password.html` and enter a test email address.
2. Submit the form, then open the Railway MySQL database `users` table.

Expected Result: The screen displays a generic confirmation message (preventing email enumeration). In MySQL, `Reset_Token` stores only a 64-character SHA-256 hash, and `Reset_Token_Expiry` is set to exactly 1 hour. Once reset, it clears to `NULL`.

What to say: *"We generate 32 random bytes, store only the SHA-256 hash in MySQL with a 1-hour expiration, and reset it to NULL upon single use."*

#3 Secure Password Handling: bcrypt with 12 Salt Rounds — RETAIN

Verification: Railway MySQL Console / phpMyAdmin / DBeaver

How to Test:
1. Open Railway MySQL Query tab and run: `SELECT Email, Password FROM users LIMIT 3;`
2. Inspect the password column string format.

Expected Result: Every password begins with `$2b$12$...`. This proves standard Modular Crypt Format using bcrypt at 12 salt rounds. Plaintext is never stored.

What to say: *"We use bcrypt (^6.0.0) with BCRYPT_SALT_ROUNDS = 12. Plaintext passwords are never saved, and hashes are omitted from all API outputs."*

CATEGORY B: SESSION SECURITY

#4 Session Fixation Defense: ID Regeneration on Login & Logout Destroy — RETAIN

Target URL: `/login.html` | `DevTools Application > Cookies`

How to Test:
1. Press `F12` > **Application** > **Cookies** > note the `connect.sid` value.
2. Log in with your MIS or Faculty account.
3. Check `connect.sid` again. Then click **Logout**.

Expected Result: The `connect.sid` cookie value changes completely after login (preventing fixation). After logout, the cookie is deleted and destroyed on the server.

What to say: *"Our auth controller runs req.session.regenerate() upon successful authentication and req.session.destroy() upon logout."*

#5 Cookie Protection: HttpOnly, Secure & SameSite Flags — RETAIN

Verification: Browser DevTools Console & Application tab

How to Test:
1. In DevTools Cookies tab, check the columns for `connect.sid`.
2. Switch to the **Console** tab and run: `console.log(document.cookie);`

Expected Result: DevTools shows `HttpOnly = ✓`, `Secure = ✓`, and `SameSite = Lax`. The console prints an empty string or omits `connect.sid` (XSS theft blocked).

What to say: *"Our session cookies have HttpOnly and Secure flags enabled, meaning malicious JavaScript cannot steal user session tokens."*

#6 Session Lifecycle: 10-Minute Rolling Inactivity Timeout & HSTS

RETAIN

Verification: DevTools Network Headers & Idle State Test

How to Test:
1. In DevTools Network tab, inspect any response header for `Strict-Transport-Security`.
2. Log in to dashboard, remain inactive for >10 minutes, then click any action.

Expected Result: Response header contains `Strict-Transport-Security: max-age=15552000`. After 10 minutes of inactivity, API returns `401 SESSION_EXPIRED`.

What to say: *"We enforce 180-day HSTS for transport security and a 10-minute idle inactivity timeout in auth middleware to prevent unattended station takeovers."*

CATEGORY C: ACCESS CONTROL

#7 Role-Based Access Control (RBAC): Faculty, IT Head, MIS & OJT — RETAIN

Verification: Browser DevTools Console (as Faculty)

How to Test:
1. Log in with a **Faculty** account.
2. In DevTools Console, execute an administrative call:

```
fetch('/api/mis-staff', { method: 'POST', headers: {'Content-Type':'application/json'}, body: '{}' }).then(r => alert('Status: ' + r.status));
```

Expected Result: Alert shows `Status: 403` Forbidden. The server checks the live database role on every request and blocks unauthorized privilege escalation.

What to say: *"Our requireRole middleware queries live MySQL data on every single API request, rejecting unauthorized roles with HTTP 403."*

#8 Privileged Actions vs. Public Reporting Separation — RETAIN

Target URL: `/public/report.html vs. /api/faculty`

How to Test:
1. Submit a valid fault report on `/public/report.html` without logging in.
2. In the same unauthenticated browser, try calling:

```
fetch('/api/faculty/add', { method: 'POST' }).then(r => alert('Status: ' + r.status));
```

Expected Result: Public fault report succeeds. Administrative call returns `Status: 401 Unauthorized`. Proves strict segregation between public and admin routes.

What to say: *"We keep student kiosks public to eliminate reporting friction, while all administrative management routes strictly enforce sessions and roles."*

CATEGORY D: INPUT AND QUERY SECURITY

#9 Input Validation & ACID Transactions (withTransaction) — RETAIN

Verification: Fault Report Form & Automated Test Script

How to Test:
1. In terminal, run: `node tests/test-submit-pc-report-validation.js`
2. Or on `/public/report.html`, try submitting with empty issue description or invalid room numbers.

Expected Result: Test suite passes all validation checks. Backend returns `HTTP 400 Bad Request`. Operations use database transactions so incomplete submissions roll back completely.

What to say: *"We validate all inputs on the backend with regex and enums, and wrap multi-step operations in database transactions to maintain ACID consistency."*

#10 SQL Injection Defense: 100% Parameterized Queries — RETAIN

Target URL: `/login.html`

How to Test:
1. On login page, enter email: `' OR '1'='1' --` and any password.
2. Click **Sign In**.

Expected Result: Rejects with "Invalid email or password" (HTTP 401). No SQL syntax errors or data dumps. mysql2 treats input strictly as literal string values via ? placeholders.

What to say: *"Every query across all repositories uses parameterized queries with question-mark placeholders, eliminating SQL injection by design."*

CATEGORY E: DATA PROTECTION AND PRIVACY

#11 Sensitive Secret Protection: Git Exclusions & Railway Variables — RETAIN

Verification: Git Terminal & Railway Environment Tab

How to Test:
1. In VS Code terminal, run: `git ls-files .env LabSync_ESP32/secrets.h`
2. Open Railway Dashboard > **Variables** tab.

Expected Result: Git command prints nothing (confirming secret files are untracked). Railway Variables tab proves production secrets are injected securely via cloud runtime environment.

What to say: *"Credentials and IoT secrets are excluded from version control via .gitignore and injected strictly through environment variables."*

#12 Data Minimization & Automated 1-Year Log Retention Purge

RETAIN

Verification: Automated Test Script: `node tests/test-retention.js`

How to Test:
1. Run the retention test in terminal.
2. Inspect error response by hitting an invalid API route: `/api/invalid-test`.

Expected Result: Test passes, verifying that records older than 1 calendar year are pruned automatically. Production error handler returns generic "Not Found" without leaking database tables.

What to say: *"Our activityRetentionService automatically purges occupancy records older than 1 calendar year, and our production error handler suppresses database schemas."*

CATEGORY F: DEVICE AND MESSAGE SECURITY

#13 ESP32 Device Token Authentication & Authorized Rooms

RETAIN

Verification: cURL or Postman Terminal Test

How to Test: Send a forged IoT event without bearer token:

```
curl -X POST https://labsync-bulsu-sarmiento-production.up.railway.app/api/occupancy/log -H "Content-Type: application/json" -d "{\"test\":true}"
```

Expected Result: Server rejects with `HTTP 401 Unauthorized` (or 403 for unauthorized room). Only the physical ESP32 sending the registered Bearer token is allowed.

What to say: *"The ESP32 uses Bearer tokens verified against iot_devices, and authorizedRooms ensures devices can only report for their assigned rooms."*

#14 Hardware Resilience: 40-ms Sensor Debounce & 15-s Heartbeat — RETAIN

Verification: Arduino Serial Monitor (115200 baud) & Automated Firmware Tests

How to Test:
1. Connect ESP32 to USB and open Serial Monitor.
2. Or run automated firmware tests: `node tests/test-firmware-heartbeat-starvation.js`

Expected Result: Firmware logs `[NetWorker Heartbeat] ✓ Success: HTTP 200 ... FreeHeap: ...` every 15 seconds. Pulling a key triggers 40ms debounce with 16-sample ADC averaging.

What to say: *"Our firmware uses 40ms debouncing with 16-sample ADC averaging to filter contact noise on physical keys, and transmits telemetry every 15s."*

CATEGORY G: AUDIT LOGGING AND SAFE ERRORS

#15 Dual Security Audit Trail: occupancy_log + audit_logs — RETAIN

Target URL: `/mis-logs.html` | `Physical ESP32 Dock`

How to Test:
1. Pull a physical key from slot 203 or 204 on the ESP32 dock without scanning a faculty QR.
2. Open `/mis-logs.html` in the MIS dashboard and refresh.

Expected Result: An immediate record appears showing `UNAUTHORIZED_REMOVAL` with exact timestamp, room number, and device ID. No deletion endpoints exist.

What to say: *"We maintain forensic accountability via occupancy_log for hardware key actions and audit_logs for administrative/login events."*

CATEGORY H: AVAILABILITY, BACKUP AND RECOVERY

#16 Service Continuity: MySQL Connection Pooling & Health Probes — RETAIN

Target URL: `/api/health` | `/api/health/db`

How to Test:
1. Open `https://labsync-bulsu-sarmiento-production.up.railway.app/api/health` in browser.
2. Next open `/api/health/db`.

Expected Result: Both return `HTTP 200 {"status":"ok","database":"connected"}` with live server uptime. Proves connection pool is healthy and active.

What to say: *"We use a 10-connection pool to prevent database socket exhaustion, while cloud uptime is managed by Railway."*

#17 Backup and Recovery Controls: Reclassified to Cloud Platform — REMOVE FROM CODE

Verification: Railway Project Console > MySQL Service > Backups tab

How to Test:
1. Log in to Railway project dashboard.
2. Click on the **MySQL** service card > open the **Backups** tab.

Expected Result: Displays automated daily cloud volume snapshots and restore points. Proves backups exist at the cloud infrastructure layer rather than app code.

What to say: *"We removed backups from our application codebase because we don't write custom cron scripts; backups are handled automatically by Railway cloud volume snapshots."*

#18 Dashboard Auto-Refresh: 3-Second Polling & Heartbeat Telemetry — RETAIN

Target URL: `/mis-keys.html` | `DevTools Network Tab`

How to Test:
1. Open `/mis-keys.html`, press `F12` > **Network** tab > filter by `Fetch/XHR`.
2. Watch requests appear every 3 seconds to `/api/keys`.
3. Lift a key from the dock: card status updates in 3 seconds automatically.

Expected Result: Network log confirms polling every 3000ms. Dashboard cards reflect physical key state changes in real time without refreshing the page.

What to say: *"3-second polling is an interface refresh loop so staff see live key movements, while hardware diagnostics are passively reported via ESP32 heartbeat telemetry."*

ADDITIONAL SECURITY FEATURES (SELECTED ADDITIONS)

A1 Student ID QR Cryptographic Verification & Replay Prevention

ADD

Target URL: `/public/report.html`

How to Test:
1. Scan a BulSU Student ID barcode/QR to generate a verification session.
2. Complete and submit the fault report (succeeds).
3. Immediately attempt to submit a second report with that identical capture session.

Expected Result: Second submission returns `HTTP 403 Forbidden: Student ID verification session has already been used`. Proves single-use nonce consumption prevents replay.

What to say: *"We generate ephemeral HMAC SHA-256 tokens with a 16-byte random nonce and 10-minute expiry; tokens are atomically marked Used_At = NOW() in MySQL upon submission, preventing replay."*

A2 API Rate Limiting & Anti-Spam Abuse Controls

ADD

Target URL: `/login.html`

How to Test:
1. On login page, type a wrong password.
2. Click **Sign In** rapidly 11 times.

Expected Result: On the 11th click, the server returns `HTTP 429 Too Many Requests: "Too many login attempts. Please try again after 15 minutes."`

What to say: *"We use express-rate-limit to enforce tiered request throttlers on login, password recovery, and student reports, blocking automated brute-force abuse."*